



Introduction to Array

Md Nazirulhasan Shawon

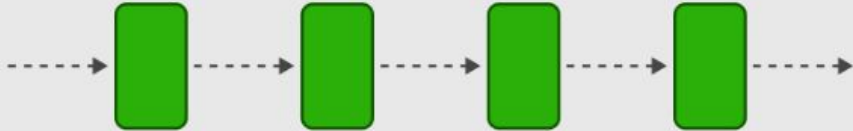
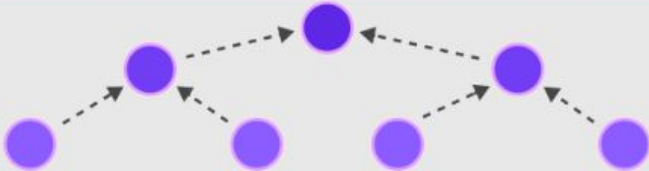
Subah Nawar

2 Data Structure

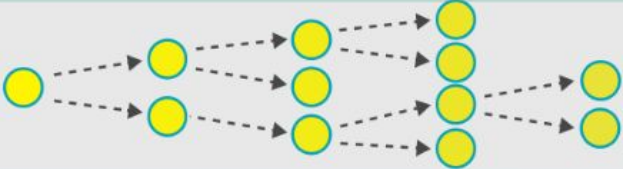
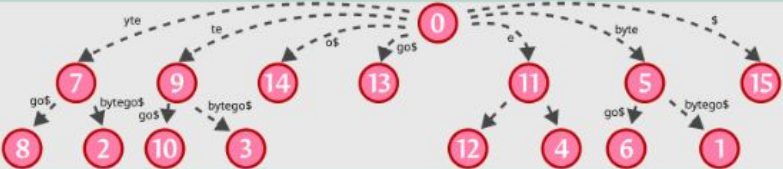
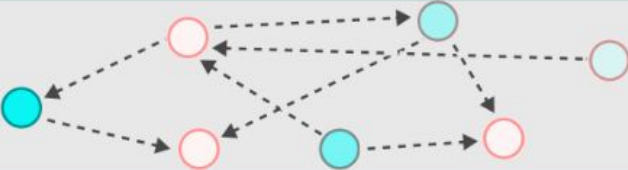
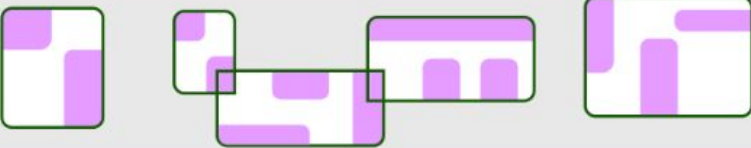
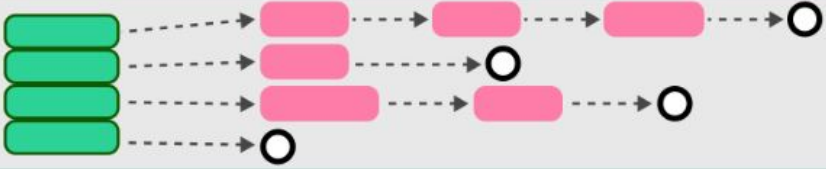


- Way of organizing and storing data in a computer.
 - **Access** and **Use** data efficiently.
- May refer to logical or mathematical representation of data.
 - Or actual implementation in computer.

3 Data Structures

Data Structure	Illustration	Use Cases
List		Twitter feeds
Array		Math operations Large data sets
Stack		Undo/Redo of word editor
Queue		Printer jobs User actions in game
Heap		Task scheduling

4 Data Structures

Tree		HTML document AI decision
Suffix Tree		Search string in document
Graph		Friendship tracking Path finding
R-tree		Nearest neighbour
Hash Table		Caching systems

5 Classification of Data Structure

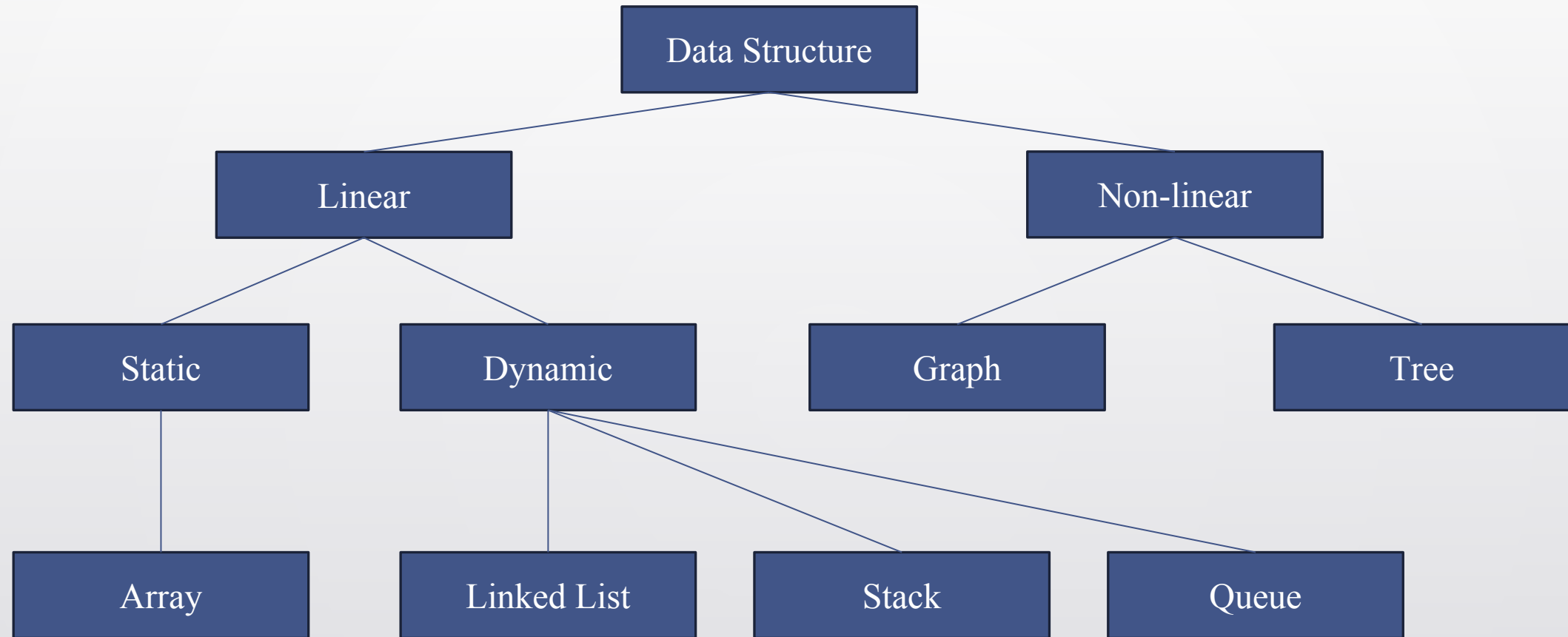
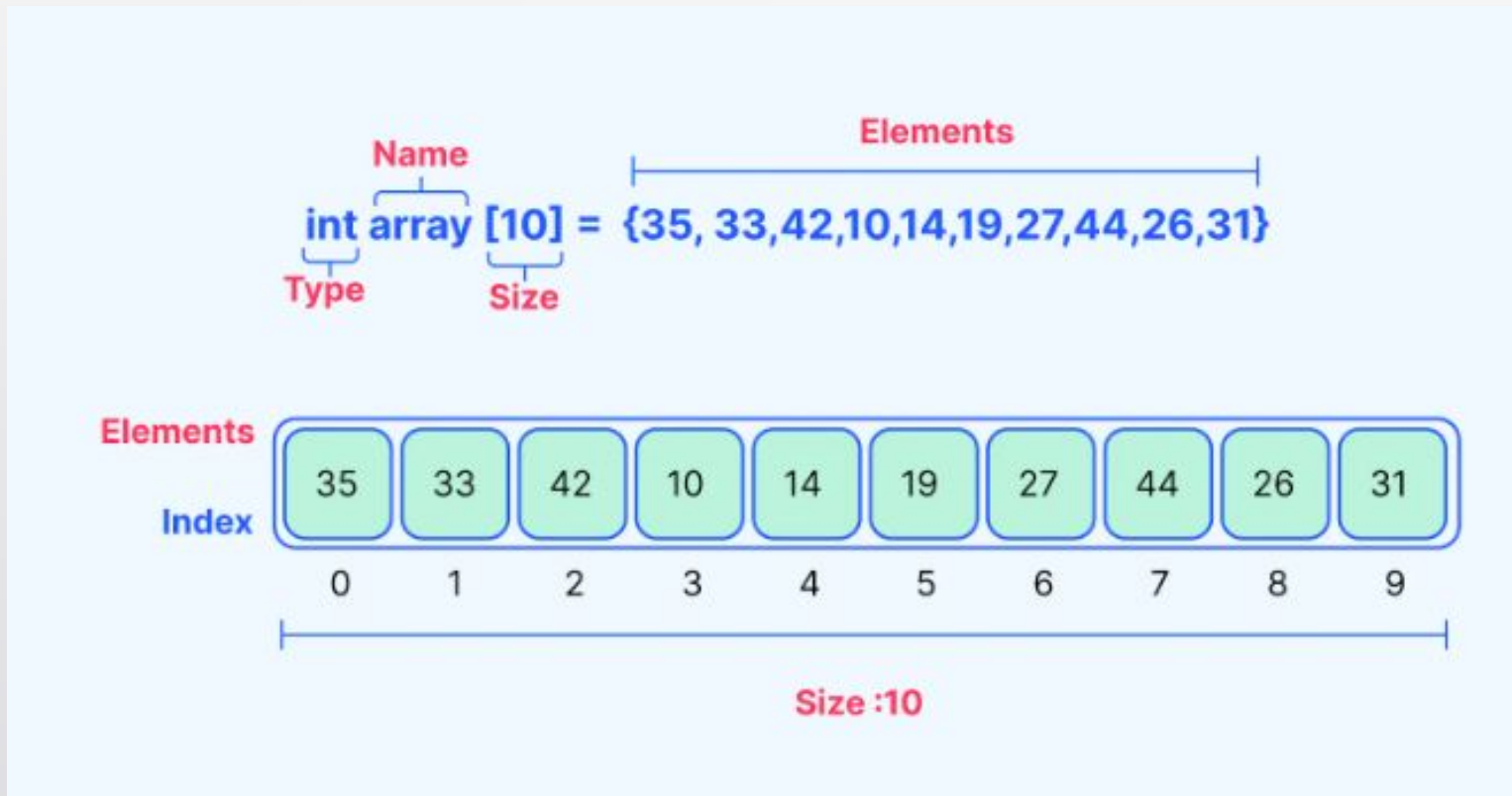


Fig 1: Classification of Data Structures

6 Array

- Collection of memory of **same data type** in contiguous memory.
- Linear data structure.



Memory Address	
31	$1000+4*9$
26	$1000+4*8$
44	$1000+4*7$
27	$1000+4*6$
19	$1000+4*5$
14	$1000+4*4$
10	$1000+4*3$
42	$1000+4*2$
33	$1000+4*1$
35	1000

7 Operations on Array



- Create
- Traverse
- Update
- Search
- Insert
- Delete

8 Array Creation

- Array declaration in C/C++

```
data_type variable_name [size];
```

- Array Initialization

```
data_type variable_name[size] = {};
```

```
data_type variable_name[size] = { $x_0, x_1, \dots, x_n$ };
```


9 Array Creation



- Array Initialization

```
int arr[]={3,4,5,6,7};
```

```
int test[2][3] = {{2, 4, 5}, {9, 0, 19}};
```

	0	1	2
0	2	4	5
1	9	0	19

10 Pointer



```
int *ptr;
```

```
int arr[5];
```

```
// store the address of the first element of arr in ptr
```

```
ptr = arr;
```

Same as before,

```
int *ptr;
```

```
int arr[5];
```

```
ptr = &arr[0];
```

11 Array Traverse

- Traverse means – travel through an area.
- In array – access each element.
- Print the array elements to show we visited it.
- Use loops to traverse any array.



```
cout<<arr[index];
```

12 Search in Array



- Check if a value exist in the array.
- We can use the traverse method.
- Access each element and check if its equal to the desired value.

13 Update in Array

- Given index 'i' and a value x
 - Update the element at index 'i' with value x.
- Just access the array index and assign new value.

14 Insertion in Array



- Given an index 'i' and a value 'x'
 - Insert the value 'x' in the 'i'th index.
 - All value after index 'i' should be shifted 1 index right.

15 Insertion in Array (Cont.)



0	1	2	3	4	5	6	7
3	7	6	9	5	9	4	2

Insert 1 at 4'th index

3	7	6	9	1	5	9	4	2
---	---	---	---	---	---	---	---	---

16 Insertion in Array (Cont.)



Procedure

- Increase the size of the array by 1.



- Shift all the element from index 'i' **1 index to the right.**



- Place element x in the desired index.



17 Deletion in Array



- Given a index 'i'
 - Delete the elements in the index 'i'
 - All values after index 'i' should be **shifted one index left.**

18 Deletion in Array (Cont.)



Delete 4'th index



19 Deletion in Array (Cont.)



Procedure

- Shift all the elements from index 'i+1' one index left.

3	7	6	9	5	9	4	2
---	---	---	---	---	---	---	---

- Reduce the size of the array by 1.

3	7	6	9	9	4	2
---	---	---	---	---	---	---

Thank You

21 Lab Report Instruction



- Theoretical portion: Underlying theory/description of array and its various operations with pseudocode/algorithm
- Methodology Portion: Implementation of each of the operations discussed in lab, you can include printed copy of your implementation done in the editor
- Also include the lab task problem of your group with proper statement and its solution approach
- Discussion Portion: Algorithm Analysis

22 More Lab Instructions



- Any assistance from AI Assistant software, electronic devices such as smartphone are strictly prohibited
- STLs are not allowed in this lab course